



UNIVERSITÀ POLITECNICA DELLE MARCHE

Corso di Programmazione Mobile

Piggy Bank

RICCARDO ANGELINI 1097724

6 gennaio 2024

Indice

1	Introduzione	3
2	Sviluppo su Android	3
2.1	Requisiti funzionali	3
2.1.1	Requisiti utente	3
2.1.2	Requisiti amministratore	4
2.1.3	Requisiti Sistema	4
2.2	Struttura dei requisiti	4
2.2.1	RF Utente	4
2.2.2	RF Amministratore	5
2.3	Casi d'Uso	5
2.3.1	CU1-Registrazione	5
2.3.2	CU2-Login	6
2.3.3	CU3-Creazione Gruppo	6
2.3.4	CU4-Visualizzazione e Selezione Gruppo	6
2.3.5	CU5-Visualizzazione e Navigazione Aree del Gruppo	6
2.3.6	CU6-Visualizzazione Lista dei Membri	7
2.3.7	CU7-Visualizzazione Saldi tra gli Utenti	7
2.3.8	CU8-Visualizzazione Lista di Spese e Specifiche	7
2.3.9	CU9-Visualizzazione Lista di Pagamenti e Specifiche	7
2.3.10	CU10-Creazione Spesa	8
2.3.11	CU11-Creazione Pagamento	8
2.3.12	CU12-Visualizzazione debiti dell'utente	8
2.3.13	CU13-Visualizzazione crediti dell'utente	8
2.3.14	CU14-Visualizzazione spese totali del gruppo	9
2.3.15	CU15- Aggiunta membro	9
2.4	Architettura	9
2.4.1	Model	9
2.4.2	ViewModel	11
2.4.3	View	12
2.5	Database	14
2.6	UI	15
2.6.1	Mockup dell'interfaccia	15
2.6.2	Rappresentazione grafica dei casi d'uso	17
2.7	Sviluppo	18
2.7.1	Adapter	18
2.7.2	Authentication	19
2.7.3	Details	19
2.7.4	Model	19
2.7.5	ViewModel	19
2.8	Testing	20
2.8.1	testIsEmailValid()	20
2.8.2	testArePasswordsEqual()	20

3	Flutter	21
3.1	Requisiti funzionali	21
3.1.1	Requisiti utente	21
3.1.2	Requisiti amministratore	21
3.1.3	Requisiti Sistema	21
3.2	Struttura dei requisiti	21
3.2.1	RF Utente	22
3.2.2	RF Amministratore	22
3.3	Casi d'Uso	22
3.3.1	CU1-Registrazione	22
3.3.2	CU2-Login	23
3.3.3	CU3-Creazione Gruppo	23
3.3.4	CU4-Visualizzazione e Selezione Gruppo	23
3.3.5	CU5-Visualizzazione e Navigazione Aree del Gruppo	24
3.3.6	CU6-Visualizzazione Lista dei Membri	24
3.3.7	CU7-Visualizzazione Lista di Spese e Specifiche	24
3.3.8	CU8-Creazione Spesa	24
3.3.9	CU9- Aggiunta membro	25
3.4	Architettura	25
3.5	Database	27
3.6	UI	27
3.6.1	Mockup	27
3.6.2	Rappresentazione grafica dei casi d'uso	27
3.7	Sviluppo	28
4	Conclusioni	29

1 Introduzione

Piggy Bank è un applicazione che permette agli utenti di gestire e monitorare le spese. Ogni utente registrato nell'applicazione ha la possibilità di creare nuovi gruppi ed essere aggiunto ad altri dai relativi amministratori. Il creatore di un gruppo ne diviene automaticamente amministratore e gode del privilegio di aggiungere nuovi membri.

Ogni utente appartenente ad un gruppo può navigare tra le seguenti aree principali: User, Expense, Payment e Report. L'architettura dell'applicazione è stata realizzata per permettere una navigazione semplice ed intuitiva da parte dell'utente; offrendo ,al contempo, una chiara separazione delle funzioni per consentire una gestione dettagliata delle spese condivise.

Ogni membro del gruppo ha accesso ai dettagli delle spese e dei pagamenti di tutto il gruppo, nonché del saldo di ogni membro. Ciò è di fondamentale importanza per garantire la trasparenza delle informazioni e dei movimenti fra i vari membri.

Piggy Bank si adatta alle esigenze specifiche di ogni utente e in diverse situazioni della vita quotidiana. Ad esempio l'applicazione può essere utilizzata: individualmente, per tenere traccia dei propri movimenti finanziari; in famiglia, per la gestione delle spese domestiche; tra amici, che vogliono monitorare e gestire le spese di una qualsiasi esperienza.

2 Sviluppo su Android

2.1 Requisiti funzionali

Di seguito viene riportata una descrizione dei requisiti dell'applicazione suddivisi per attore.

2.1.1 Requisiti utente

1. Autenticazione e registrazione utente.
2. Creazione gruppo
3. Visualizzazione e selezione gruppo
4. Visualizzazione e navigazione aree del gruppo
5. Visualizzazione lista dei membri
6. Visualizzazione lista di spese e specifiche
7. Visualizzazione lista di pagamenti e specifiche
8. Creazione spesa
9. Creazione pagamento

10. Visualizzazione debiti dell'utente
11. Visualizzazione crediti dell'utente
12. Visualizzazione spese totali del gruppo
13. Visualizzazione saldi tra gli utenti

2.1.2 Requisiti amministratore

1. Aggiunta di un membro al gruppo

2.1.3 Requisiti Sistema

1. Visualizzazione messaggi di errore
2. Gestione dei dati con collegamento al database

2.2 Struttura dei requisiti

In questa sezione vengono elencati i requisiti funzionali dell'applicazione.

2.2.1 RF Utente

Requisiti funzionali di un utente.

- **RF1- Registrazion:** il sistema dovrà permettere la creazione di un account.
- **RF2- Login:** il sistema dovrà permettere all'utente con credenziali di autenticarsi per accedere all'applicazione.
- **RF3-Creazione gruppo:** il sistema dovrà consentire ad un utente la creazione di un nuovo gruppo
- **RF4-Visualizzazione e selezione gruppo:** il sistema dovrà riportare la lista di gruppi a cui l'utente appartiene, con la possibilità di selezionarne uno.
- **RF5-Visualizzazione e navigazione aree del gruppo:** il sistema dovrà consentire all'utente di navigare tra le aree del gruppo selezionando una delle 4 immagini
- **RF6-Visualizzazione lista dei membri:** il sistema dovrà mostrare all'utente la lista dei membri del gruppo
- **RF7-Visualizzazione saldi tra gli utenti:** il sistema dovrà restituire il saldo fra i membri del gruppo
- **RF8-Visualizzazione lista di spese e specifiche:** il sistema dovrà restituire all'utente le spese del gruppo, con la possibilità di vederne le specifiche

- **RF9-Visualizzazione lista di pagamenti e specifiche:** il sistema dovrà restituire all'utente i pagamenti del gruppo, con la possibilità di vederne le specifiche
- **RF10-Creazione spesa:** il sistema dovrà permettere all'utente l'inserimento di una nuova spesa
- **RF11-Creazione pagamento:** il sistema dovrà permettere all'utente l'inserimento di un nuovo pagamento
- **RF12-Visualizzazione debiti dell'utente :** il sistema dovrà restituire i debiti dell'utente
- **RF13-Visualizzazione crediti dell'utente:** il sistema dovrà restituire i crediti dell'utente
- **RF-14Visualizzazione spese totali del gruppo:** il sistema dovrà restituire la somma delle spese totali del gruppo

2.2.2 RF Amministratore

Requisiti funzionali di un amministratore.

- **RF15-Aggiunta membro:** il sistema deve permettere all'amministratore di inserire un utente al gruppo.

2.3 Casi d'Uso

2.3.1 CU1-Registrazione

Descrizione: Questo caso d'uso permette all'utente di registrarsi nell'applicazione.

Flusso Principale:

1. L'utente avvia l'applicazione.
2. L'applicazione fornisce l'opzione di registrazione.
3. L'utente seleziona l'opzione di registrazione.
4. Il sistema fornisce i campi per la procedura di creazione di un account.
5. L'utente compila tutti i campi richiesti.
6. Il sistema verifica e registra l'utente nell'applicazione.
7. L'utente riceve la conferma della registrazione.

2.3.2 CU2-Login

Descrizione: Questo caso d'uso permette all'utente di effettuare l'accesso all'applicazione.

Flusso Principale:

1. L'utente avvia l'applicazione.
2. L'applicazione richiede le credenziali.
3. L'utente inserisce le credenziali.
4. Il sistema verifica le credenziali.
5. Se le credenziali sono valide, l'utente accede all'applicazione.

2.3.3 CU3-Creazione Gruppo

Descrizione: Questo caso d'uso permette all'utente di creare un nuovo gruppo.

Flusso Principale:

1. L'utente seleziona l'opzione per la creazione di un nuovo gruppo.
2. Il sistema richiede i dettagli del nuovo gruppo.
3. L'utente inserisce il nome e la tipologia del gruppo.
4. Il sistema verifica e crea il nuovo gruppo.
5. L'utente riceve la conferma di creazione del gruppo.
6. L'utente viene reindirizzato alla lista dei gruppi, che ora include il nuovo gruppo appena creato.

2.3.4 CU4-Visualizzazione e Selezione Gruppo

Descrizione: Questo caso d'uso permette all'utente di visualizzare la lista dei gruppi e selezionarne uno.

Flusso Principale:

1. L'utente visualizza la lista dei gruppi.
2. L'utente seleziona un gruppo dalla lista.
3. Il sistema reindirizza l'utente alle aree di gestione del gruppo.

2.3.5 CU5-Visualizzazione e Navigazione Aree del Gruppo

Descrizione: Questo caso d'uso permette all'utente di navigare tra le aree del gruppo selezionato.

Flusso Principale:

1. L'utente visualizza le immagini rappresentanti le aree del gruppo.
2. L'utente seleziona un'immagine per accedere all'area corrispondente.

2.3.6 CU6-Visualizzazione Lista dei Membri

Descrizione: Questo caso d'uso permette all'utente di visualizzare la lista dei membri del gruppo.

Flusso Principale:

1. L'utente seleziona l'area User per visualizzare la lista dei membri.
2. Il sistema mostra la lista dei membri del gruppo.

2.3.7 CU7-Visualizzazione Saldi tra gli Utenti

Descrizione: Questo caso d'uso permette all'utente di visualizzare i saldi tra i membri del gruppo.

Flusso Principale:

1. L'utente seleziona l'area User.
2. Il sistema calcola e mostra i saldi tra gli utenti del gruppo.

2.3.8 CU8-Visualizzazione Lista di Spese e Specifiche

Descrizione: Questo caso d'uso permette all'utente di visualizzare la lista di spese del gruppo e le relative specifiche.

Flusso Principale:

1. L'utente seleziona l'area Expense per visualizzare la lista di spese.
2. Il sistema mostra la lista di spese del gruppo ordinate dalla data meno recente alla più recente.
3. L'utente può selezionare una spesa per visualizzarne le specifiche.

2.3.9 CU9-Visualizzazione Lista di Pagamenti e Specifiche

Descrizione: Questo caso d'uso permette all'utente di visualizzare la lista di pagamenti del gruppo e le relative specifiche.

Flusso Principale:

1. L'utente seleziona l'opzione per visualizzare la lista di pagamenti.
2. Il sistema mostra la lista di pagamenti del gruppo dalla data meno recente alla più recente.
3. L'utente può selezionare un pagamento per visualizzarne le specifiche.

2.3.10 CU10-Creazione Spesa

Descrizione: Questo caso d'uso permette all'utente di inserire una nuova spesa.

Flusso Principale:

1. L'utente accede all'area Expense.
2. L'utente seleziona l'opzione per aggiungere una nuova spesa.
3. Il sistema richiede i dettagli della nuova spesa.
4. L'utente inserisce i dettagli della spesa.
5. Il sistema verifica e aggiunge la nuova spesa.
6. Il sistema comunica la conferma dell'inserimento.
7. Il sistema aggiorna la lista delle spese del gruppo.

2.3.11 CU11-Creazione Pagamento

Descrizione: Questo caso d'uso permette all'utente di inserire un nuovo pagamento.

Flusso Principale:

1. L'utente seleziona l'opzione per aggiungere un nuovo pagamento.
2. Il sistema richiede i dettagli del nuovo pagamento.
3. L'utente inserisce i dettagli del pagamento.
4. Il sistema verifica e aggiunge il nuovo pagamento.
5. Il sistema comunica la conferma dell'inserimento.
6. Il sistema aggiorna la lista di pagamenti del gruppo.

2.3.12 CU12-Visualizzazione debiti dell'utente

Descrizione: Questo caso d'uso permette all'utente di controllare i debiti a suo carico.

Flusso Principale:

1. L'utente seleziona l'area Reports.
2. Il sistema calcola e restituisce i debiti a suo carico.

2.3.13 CU13-Visualizzazione crediti dell'utente

Descrizione: Questo caso d'uso permette all'utente di controllare i crediti a suo carico.

Flusso Principale:

1. L'utente seleziona l'area Reports.
2. Il sistema calcola e restituisce i crediti a suo carico.

2.3.14 CU14-Visualizzazione spese totali del gruppo

Descrizione: Questo caso d'uso permette all'utente di osservare le spese totali del gruppo.

Flusso Principale:

1. L'utente seleziona l'area Reports.
2. Il sistema calcola e restituisce le spese totali del gruppo.

2.3.15 CU15- Aggiunta membro

Descrizione: Questo caso d'uso permette all'amministratore di aggiungere un utente al gruppo.

Flusso Principale:

1. L'amministratore seleziona l'area Users.
2. L'amministratore seleziona il bottone "New User".
3. Il sistema restituisce la lista di utenti non appartenenti al gruppo.
4. L'amministratore seleziona un utente.
5. Il sistema genera un alert.
6. L'amministratore conferma l'aggiunta dell'utente.
7. Il sistema aggiorna la lista di utenti non appartenenti al gruppo.

2.4 Architettura

L'architettura utilizzata per lo sviluppo dell'applicazione segue il pattern MV-VM (Model-View-ViewModel). Questa metodologia di progettazione, fortemente discussa durante il corso, facilita la manutenibilità e chiarezza del codice, grazie alla separazione delle responsabilità all'interno dell'applicazione.

2.4.1 Model

Il componente Model rappresenta la logica di business e i dati dell'applicazione. Le classi di modello definiscono la struttura dei dati utilizzati all'interno dell'applicazione. Ogni classe è stata implementata per rappresentare le entità principali coinvolte nella gestione delle spese.

La classe **User** rappresenta un utente dell'applicazione e contiene i seguenti campi:

- *id*: identificatore univoco ;
- *name*: nome dell'utente;
- *email*: indirizzo email dell'utente;

- *password*: password dell'utente;
- *userExpense*: lista di id delle spese sostenute dall'utente;
- *userPayment*: lista di id dei pagamenti sostenuti dall'utente.

La classe **Group** rappresenta un gruppo all'interno dell'applicazione ed è formata dai seguenti campi:

- *id*: identificatore univoco;
- *name*: nome del gruppo;
- *type*: tipologia del gruppo;
- *members*: lista degli utenti appartenenti al gruppo;
- *groupExpense*: lista degli id delle spese e dei pagamenti sostenuti dai membri del gruppo;
- *idAmministratore*: identificatore univoco dell'utente creatore del gruppo.

La classe **Expense** rappresenta una spesa sostenuta dagli utenti e contiene i seguenti campi:

- *id*: identificatore univoco;
- *name*: nome della spesa;
- *amount*: importo della spesa;
- *payer*: id dell'utente che effettua il pagamento;
- *debtors*: lista di id degli utenti che condividono la spesa;
- *date*: data in cui si effettua la spesa.

La classe **Payment** rappresenta una spesa sostenuta dagli utenti e contiene i seguenti campi:

- *id*: identificatore univoco;
- *name*: nome del pagamento;
- *amount*: importo del pagamento;
- *expense*: id della spesa collegata al pagamento;
- *date*: data in cui si effettua il pagamento.

2.4.2 ViewModel

Questa componente è responsabile della logica di presentazione e del coordinamento delle operazioni tra la View e il Model. Le classi legati al ViewModel sono: **GroupViewModel** e **GroupRepository**. Quest'ultima comunica con il Model per fornire dati al ViewModel; gestisce le operazioni di lettura e scrittura sul database **Firestore**. Le Figure 1a e 1b mostrano due schermate che visualizzano un anteprima della classe GroupRepository.

```

class GroupRepository {
    private val firestore: FirebaseFirestore = FirebaseFirestore.getInstance()
    private val groupCollection: CollectionReference = firestore.collection(CollectionName.GROUP)
    private val userCollection: CollectionReference = firestore.collection(CollectionName.USER)
    private val auth: FirebaseAuth = FirebaseAuth.getInstance()
    private val currentUser: FirebaseUser? = auth.currentUser

    //DEFINE GROUPS

    @Throws(IOException::class)
    suspend fun getGroupsForCurrentUser(): List<Group> {
        val userId = currentUser?.uid ?: throw UserNotLoggedInException()

        try {
            // Query per gruppi per utente e se l'utente aggiunto è gestibile
            val query = groupCollection.whereArrayContainsAny(field("users"), userId)
            val querySnapshot = query.get().await()

            val groups = mutableListOf<Group>()

            for (document in querySnapshot.documents) {
                val group = document.toObject(Group::class.java)
                group.set { group,
                    group.add(it)
                }
            }

            return groups
        } catch (e: Exception) {
            Log.e(TAG, "Error checking admin status", e)
            return false
        }
    }
}

```

(a) Class GroupRepository

```

suspend fun isCurrentUserAdmin(groupId: String): Boolean {
    try {
        val userId = currentUser?.uid ?: throw UserNotLoggedInException()
        val groupDocument = groupCollection.document(groupId).get().await()
        val adminId = groupDocument.getString(field("adminId"))

        Log.d(TAG, "GroupRepository", "MSG: AdminId for group $groupId: $adminId")

        if (userId == adminId) {
            return true
        } else {
            return false
        }
    } catch (e: Exception) {
        Log.e(TAG, "GroupRepository", "MSG: Error checking admin status", e)
        return false
    }
}

```

(b) Esempio di funzione nel repository

Figura 1: Due istantanee della classe GroupRepository.

Si può notare l'accesso a Firestore mediante le classi `FirebaseFirestore` e `CollectionReference`. Ho utilizzato il costrutto *try-catch* per la gestione degli errori e i *Log* per la loro registrazione. La classe `GroupViewModel` comunica con il Repository ed il Model fornendo i dati necessari alla View. La Figura 2 introduce la classe `GroupViewModel` implementata; la scelta dell'utilizzo dei *LiveData* ci permette di osservare aggiornamenti in tempo reale degli elementi della UI quando cambiano i dati. Inoltre l'uso di coroutine come *viewModelScope.launch* (Figura 3a) consente di eseguire operazioni asincrone in un thread separato da quello principale.

La classe **GroupViewModelFactory** (Figura 3b) si occupa di instanziare il ViewModel che viene associato all'activity o fragment.

```

10 class GroupViewModel(private val groupRepository: GroupRepository): ViewModel() {
11
12     private val _groups = MutableLiveData<List<Group>>()
13     @Nullable
14     val groups: LiveData<List<Group>>
15         get() = _groups
16
17     private val _isAdmin = MutableLiveData<Boolean>()
18     @Nullable
19     val isAdmin: LiveData<Boolean>
20         get() = _isAdmin
21
22     private val _userData = MutableLiveData<List<User>>()
23     @Nullable
24     val userData: LiveData<List<User>>
25         get() = _userData
26
27     private val _expenses = MutableLiveData<List<Expense>>()
28     @Nullable
29     val expenses: LiveData<List<Expense>>
30         get() = _expenses
31
32     private val _payments = MutableLiveData<List<Payment>>()
33     @Nullable
34     val payments: LiveData<List<Payment>>
35         get() = _payments
36
37     private val _userName = MutableLiveData<String>()
38     @Nullable
39     val userName: LiveData<String>
40         get() = _userName
41 }

```

Figura 2: Classe GroupViewModel

```

//Metodo GROUP
//Metodo per la lista di gruppi
@Nullable
fun loadGroups() {
    viewModel.groupRepository.loadGroups() {
        try {
            val groups = groupRepository.getGroupsForCurrentUser()
            _groups.postValue(groups)
        } catch (e: Exception) {
            Log.e(TAG, "Errore durante il caricamento dei gruppi")
        }
    }
}

@Nullable
fun addNewGroup(name: String, type: String) {
    viewModel.groupRepository.addNewGroup(name, type) {
        try {
            groupRepository.addNewGroup(name, type)
            loadGroups()
        } catch (e: Exception) {
            throw GroupAdditionException("Errore durante l'aggiunta del gruppo")
        }
    }
}

```

(a) Esempio di funzione nel view-model

```

//Implementazione della classe GroupViewModel
@Nullable
fun loadGroups() {
    viewModel.groupRepository.loadGroups() {
        try {
            val groups = groupRepository.getGroupsForCurrentUser()
            _groups.postValue(groups)
        } catch (e: Exception) {
            Log.e(TAG, "Errore durante il caricamento dei gruppi")
        }
    }
}

@Nullable
fun addNewGroup(name: String, type: String) {
    viewModel.groupRepository.addNewGroup(name, type) {
        try {
            groupRepository.addNewGroup(name, type)
            loadGroups()
        } catch (e: Exception) {
            throw GroupAdditionException("Errore durante l'aggiunta del gruppo")
        }
    }
}

```

(b) Class GroupViewModelFactory

Figura 3: Implementazione del ViewModel.

2.4.3 View

Questa componente è responsabile della presentazione dei dati e dell'interazione con l'utente. Le viste raccolgono e visualizzano i dati passati dal ViewModel e li rendono visibili all'utente. I file di layout XML definiscono quali View vengono utilizzate e le loro proprietà, dopodiché vengono richiamati nei Fragment e nelle Activity dell'applicazione. Nell'implementazione dell'applicazione Piggy Bank, abbiamo adottato vari tipi di layout, tra cui Linear, Relative, Grid e Constraint Layout. Le componenti di layout includono Text View, Buttons e diversi Input Controls come Edit Text, Spinner e Checkbox.

Di seguito sono elencate le classi Fragment e Activity dell'applicazione:

- MainActivity
- LoginFragment
- RegisterFragment
- ListGroupFragment

- CreateGroupFragment
- DetailsGroupFragment
- AddExpenseFragment
- ExpenseFragment
- PayerFragment
- GraphFragment
- AddUserFragment
- UserFragment

All'interno di alcuni Fragment si adoperano strumenti di Data Binding (Figura 4) per collegare dinamicamente i dati del ViewModel con gli elementi dell'interfaccia utente.

```

A Recyclerview
class CreateGroupFragment : android.support.design.widget.AppCompatActivity() {
    private lateinit var binding: FragmentCreateGroupBinding
    private val groupViewModel: GroupViewModel by viewModels() {
        GroupViewModelFactory(
            GroupRepository() // Create your instances of repositories here
        )
    }
}

A Recyclerview
or
override fun onCreateView(
    inflater: LayoutInflater, container: ViewGroup?,
    savedInstanceState: Bundle?
): View? {
    binding = FragmentCreateGroupBinding.inflate(inflater, container, attachToRoot = false)

    binding.lifecycleOwner = viewLifecycleOwner
    binding.viewModel = groupViewModel
    binding.createGroupButton.setOnClickListener { view ->
        val groupName = binding.groupName.text.toString()
        val groupType = binding.groupType.text.toString()

        // Assicurati che i valori non siano vuoti e nonli prima di chiamare addGroup
        if (groupName.isEmpty() || groupType.isEmpty()) {
            GroupViewModel.addGroup(groupName, groupType)
            requireActivity().supportFragmentManager.popBackStack()
        } else {
            // Gestisci il caso in cui uno o entrambi i campi sono vuoti
        }
    }
}

```

Figura 4: Classe CreateGroupFragment

L'applicazione presenta diverse RecyclerView; ciò rispecchia la necessità di poter visualizzare elenchi dinamici nei vari contesti come: la lista dei gruppi, degli utenti, delle spese e dei pagamenti. La Figura 5 espone l'implementazione di diversi **adapter** e **listitem** per soddisfare le funzionalità richieste dall'app e la realizzazione di liste con elementi cliccabili e non cliccabili.



(a) Classe CustomAdapter

(b) file XML itemviewgroup

Figura 5: Implementazione RecyclerView.

2.5 Database

In questo paragrafo illustreremo l'utilizzo del servizio **Firestore** per lo sviluppo autonomo di funzionalità di backend quali **Firebase Authentication** e **Firebase Firestore**. Firebase Authentication offre un'interfaccia per gestire l'autenticazione degli utenti in modo sicuro. L'applicazione consente di registrarsi inserendo il proprio indirizzo e-mail e la propria password. La Figura 6 mostra una schermata che visualizza alcuni utenti registrati nell'applicazione.

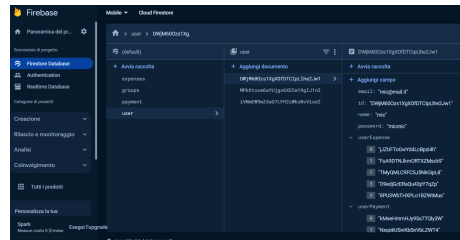


Figura 6: Schermata utenti registrati in Firebase Authentication

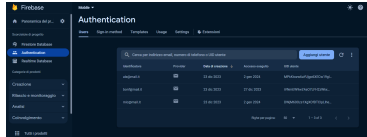
Firebase Firestore è un database No-SQL che organizza i dati in documenti e collezioni. La struttura del database dell'applicazione (Figura 7a) è formata dalle seguenti raccolte principali:

- *user*: contiene documenti che rappresentano gli utenti registrati;
- *groups*: contiene documenti che rappresentano i gruppi creati dagli utenti;
- *expenses*: contiene i documenti che rappresentano le spese create dagli utenti all'interno di un gruppo;
- *payment*: contiene i documenti che rappresentano i pagamenti sostenuti dagli utenti all'interno del gruppo.

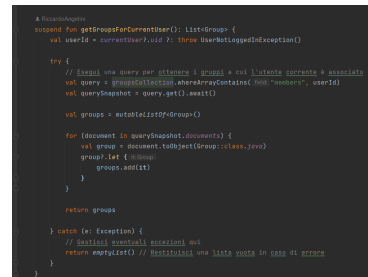
Le raccolte coincidono con le classi del nostro modello dei dati ed i documenti contengono gli attributi delle classi. L'integrazione di Firebase nelle classi del nostro Model semplifica il processo di gestione dei dati offrendo diversi vantaggi come:

- **semplificazione del codice:** operazioni di lettura e scrittura più chiare e concise ;
- **risposte in tempo reale:** aggiornamenti istantanei nella nostra applicazione, ogni volta che i dati cambiano nel db;
- **modellazione intuitiva:** la struttura del database firestore si adatta perfettamente alla modellazione delle classi di modello;

La figura 7b mostra una funzione nel repository dove viene effettuata una query firestore nella collezione group. L'obiettivo della funzione "getGroupsFoCurrentUser" è recuperare la lista di gruppi, in cui l'id dell'utente corrente è contenuto nel campo members.



(a) Raccolte dei documenti in Cloud Firestore.



(b) Esempio di query Firestore nel repository.

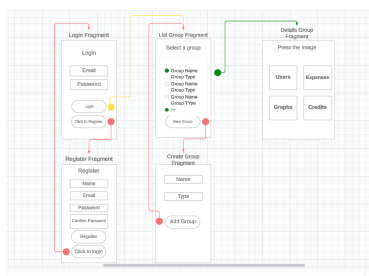
Figura 7: Firebase Firestore.

2.6 UI

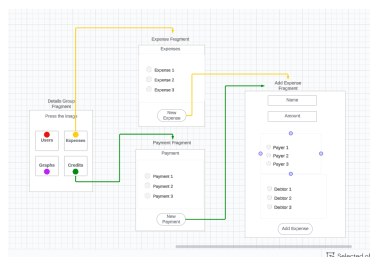
In questa sezione presentiamo le anteprime del mockup dell'interfaccia, create con il software Lucidchart, insieme a una rappresentazione grafica dei casi d'uso.

2.6.1 Mockup dell'interfaccia

La Figura 8a mostra i frammenti di Login e Registrazione che costituiscono la schermata iniziale dell'applicazione. Dopo l'autenticazione, l'utente accede al fragment di scelta del gruppo, con la possibilità di poterne creare uno nuovo. Una volta selezionato un gruppo si accede alla schermata di gestione (Figura 8b) che presenta 4 immagini cliccabili: Users, Expenses, Payments e Report. Questa schermata permette all'utente di navigare tra le 4 aree di gestione del gruppo. Selezionando l'area Expense o Payment si reindirizza l'utente a due diverse schermate che presentano una lista cliccabile di spese e pagamenti. In entrambi i casi l'utente accede alla schermata di inserimento di una spesa/pagamento selezionando il bottone "New Expense" o "New Payment". L'area di gestione utenti (Figura 9a) visualizza la lista dei membri del gruppo con il relativo



(a) Fragment di autenticazione



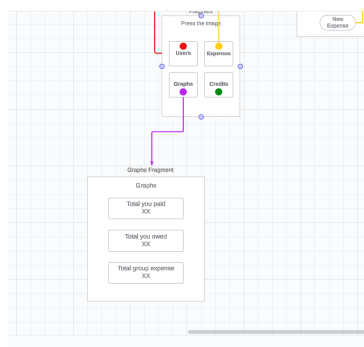
(b) Area di gestione delle spese e dei pagamenti

Figura 8: Prime schermate dell'applicazione.

bilancio. L'admin ha la possibilità di aggiungere nuovi membri accedendo al frammento "Add User". Infine, la Figura 9b mostra l'accesso all'ultima delle 4 aree disponibili, il Report. Questi mockup rappresentano la disposizione e la struttura dell'interfaccia utente.



(a) Area di gestione utenti



(b) Area report

Figura 9: Ultimi frammenti dell'applicazione.

2.6.2 Rappresentazione grafica dei casi d'uso

In questo paragrafo verranno esposti alcuni tra i casi d'uso più rilevanti dell'applicazione.

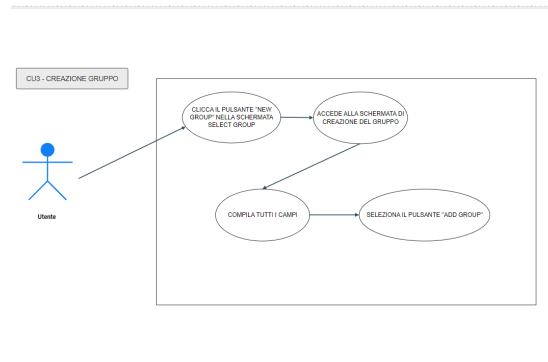


Figura 10: CU3-Creazione del gruppo

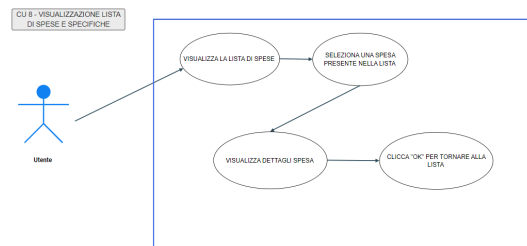


Figura 11: CU-8 Visualizzazione lista di spese e specifiche

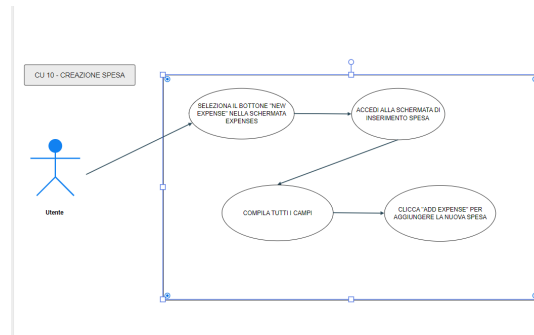


Figura 12: CU-10 Creazione spesa

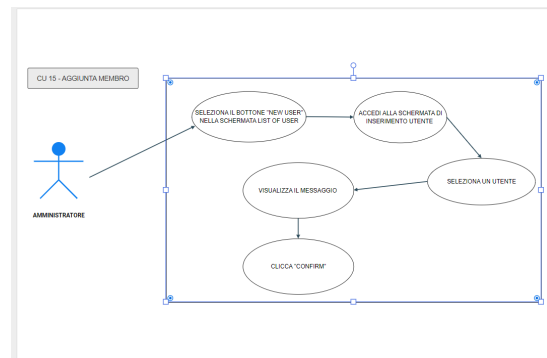


Figura 13: CU-15 Aggiunta membro

2.7 Sviluppo

L'implementazione dell'app è avvenuta per Fragment riflettendone la struttura. Si è partiti con l'implementazione del processo di Login e di Registrazione per poi passare alla gestione (visualizzazione e creazione) dei gruppi. Successivamente ciascuna area del gruppo è stata sviluppata separatamente, partendo dalla sezione User e concludendo con il Report. Prima di procedere alla realizzazione di una nuova schermata si deve verificare il corretto funzionamento della precedente. Questa metodologia **step by step** permette una buona validazione delle funzionalità implementate e facilita eventuali operazioni di debugging.

L'applicazione è stata suddivisa in package per organizzare le classi, nei prossimi sottoparagrafi andremo ad illustrarli separatamente.

2.7.1 Adapter

Include le classi per gestire la logica delle RecyclerView:

- AddUserListAdapter
- CustomAdapter

- ExpenseListAdapter
- PaymentListAdapterSpinnerUserAdapter
- SplitUserAdapter
- UserListAdapter

2.7.2 Authentication

Racchiude le classi per il processo di autenticazione e sign-in:

- LoginFragment
- RegisterFragment

2.7.3 Details

Include le classi che implementano le aree dei gruppo:

- AddExpenseFragment
- AddUserFragment
- ExpenseFragment
- GraphFragment
- PayerFragment
- UserFragment

2.7.4 Model

Contiene le classi di modello e il Repository:

- Expense
- Group
- GroupRepository
- Payment
- User

2.7.5 ViewModel

: Include le classi per gestire la logica dell'applicazione.

- GroupViewModel
- GroupViewModelFactory

2.8 Testing

In questo sottoparagrafo viene illustrata la fase di testing dell'applicazione. Nella classe `ExampleInstrumentedTest` sono stati implementati alcuni test per identificare il riscontro di eventuali problematiche del codice. Di seguito mostreremo in dettaglio i test effettuati.

2.8.1 `testIsEmailValid()`

Questo test verifica il corretto funzionamento della funzione `isEmailValid` all'interno della classe `RegisterFragment`. Tale funzione deve accertarsi che la stringa passata come parametro soddisfi il formato definito dell'espressione regolare (*Regex*), a quel punto restituisce un valore booleano. In sostanza, se l'indirizzo e-mail fornito è valido, restituisce `True`, altrimenti restituisce `False`.

A screenshot of a code editor showing a Java test method. The code is as follows:

```
17 // Test
18 @Test
19 void testIsEmailValid() {
20     // Password valida
21     assertTrue(RegisterFragment.isEmailValid("password123", "password123"))
22
23     // Password invalida
24     assertFalse(RegisterFragment.isEmailValid("password123", "password124"))
25 }
```

Figura 14: Funzione di testing `testIsEmailValid()`

Possiamo visualizzare i seguenti test su 4 formati email differenti:

1. **test@example.com**: L'email passata alla funzione soddisfa l'espressione regolare.
2. **test@.com**: L'email passata alla funzione **non** soddisfa l'espressione regolare, perchè dopo la **@** deve seguire almeno una lettera.
3. **test@example**: L'email passata alla funzione **non** soddisfa l'espressione regolare, perchè deve avere almeno due lettere nel dominio.
4. **test@.**: L'email passata alla funzione **non** soddisfa l'espressione regolare, perchè non può iniziare con un punto dopo la **@**.

2.8.2 `testArePasswordsEqual()`

Questo test verifica il corretto funzionamento della funzione `arePasswordsEqual` all'interno della classe `RegisterFragment`. Tale funzione deve accertarsi che le due stringhe passate come parametro coincidano, in quel caso restituisce `True`, altrimenti `False`. Possiamo visualizzare due casi:

A screenshot of a code editor showing a Java test method. The code is as follows:

```
17 // Test
18 @Test
19 void testArePasswordsEqual() {
20     // Password valida
21     assertTrue(RegisterFragment.arePasswordsEqual("password123", "password123"))
22
23     // Password diversa
24     assertFalse(RegisterFragment.arePasswordsEqual("password123", "password124"))
25 }
```

Figura 15: Funzione di testing `testArePasswordsEqual()`

1. Le due stringhe passate come parametro coincidono (*password123*, *password123*)
2. Le due stringhe passate come parametro **non** coincidono (*password123*, *password456*)

3 Flutter

3.1 Requisiti funzionali

Di seguito viene riportata una descrizione dei requisiti dell'applicazione in flutter suddivisi per attore.

3.1.1 Requisiti utente

1. Autenticazione e registrazione utente.
2. Creazione gruppo
3. Visualizzazione e selezione gruppo
4. Visualizzazione e navigazione aree del gruppo
5. Visualizzazione lista dei membri
6. Visualizzazione lista di spese e specifiche
7. Creazione spesa

3.1.2 Requisiti amministratore

1. Aggiunta di un membro al gruppo

3.1.3 Requisiti Sistema

1. Visualizzazione messaggi di errore
2. Gestione dei dati con collegamento al database

3.2 Struttura dei requisiti

In questa sezione vengono elencati i requisiti funzionali dell'applicazione.

3.2.1 RF Utente

Requisiti funzionali di un utente.

- **RF1- Registrazione:** il sistema dovrà permettere la creazione di un account.
- **RF2- Login:** il sistema dovrà permettere all'utente con credenziali di autenticarsi per accedere all'applicazione.
- **RF3-Creazione gruppo:** il sistema dovrà consentire ad un utente la creazione di un nuovo gruppo
- **RF4-Visualizzazione e selezione gruppo:** il sistema dovrà riportare la lista di gruppi a cui l'utente appartiene, con la possibilità di selezionarne uno.
- **RF5-Visualizzazione e navigazione aree del gruppo:** il sistema dovrà consentire all'utente di navigare tra le aree del gruppo selezionando una delle 2 immagini
- **RF6-Visualizzazione lista dei membri:** il sistema dovrà mostrare all'utente la lista dei membri del gruppo
- **RF7-Visualizzazione lista di spese e specifiche:** il sistema dovrà restituire all'utente le spese del gruppo, con la possibilità di vederne le specifiche
- **RF8-Creazione spesa:** il sistema dovrà permettere all'utente l'inserimento di una nuova spesa

3.2.2 RF Amministratore

Requisiti funzionali di un amministratore.

- **RF9-Aggiunta membro:** il sistema deve permettere all'amministratore di inserire un utente al gruppo.

3.3 Casi d'Uso

3.3.1 CU1-Registrazione

Descrizione: Questo caso d'uso permette all'utente di registrarsi nell'applicazione.

Flusso Principale:

1. L'utente avvia l'applicazione.
2. L'applicazione fornisce l'opzione di registrazione.
3. L'utente seleziona l'opzione di registrazione.

4. Il sistema fornisce i campi per la procedura di creazione di un account.
5. L'utente compila tutti i campi richiesti.
6. Il sistema verifica e registra l'utente nell'applicazione.
7. L'utente riceve la conferma della registrazione.

3.3.2 CU2-Login

Descrizione: Questo caso d'uso permette all'utente di effettuare l'accesso all'applicazione.

Flusso Principale:

1. L'utente avvia l'applicazione.
2. L'applicazione richiede le credenziali.
3. L'utente inserisce le credenziali.
4. Il sistema verifica le credenziali.
5. Se le credenziali sono valide, l'utente accede all'applicazione.

3.3.3 CU3-Creazione Gruppo

Descrizione: Questo caso d'uso permette all'utente di creare un nuovo gruppo.

Flusso Principale:

1. L'utente seleziona l'opzione per la creazione di un nuovo gruppo.
2. Il sistema richiede i dettagli del nuovo gruppo.
3. L'utente inserisce il nome e la tipologia del gruppo.
4. Il sistema verifica e crea il nuovo gruppo.
5. L'utente riceve la conferma di creazione del gruppo.
6. L'utente viene reindirizzato alla lista dei gruppi, che ora include il nuovo gruppo appena creato.

3.3.4 CU4-Visualizzazione e Selezione Gruppo

Descrizione: Questo caso d'uso permette all'utente di visualizzare la lista dei gruppi e selezionarne uno.

Flusso Principale:

1. L'utente visualizza la lista dei gruppi.
2. L'utente seleziona un gruppo dalla lista.
3. Il sistema reindirizza l'utente alle aree di gestione del gruppo.

3.3.5 CU5-Visualizzazione e Navigazione Aree del Gruppo

Descrizione: Questo caso d'uso permette all'utente di navigare tra le aree del gruppo selezionato.

Flusso Principale:

1. L'utente visualizza le immagini rappresentanti le aree del gruppo.
2. L'utente seleziona un'immagine per accedere all'area corrispondente.

3.3.6 CU6-Visualizzazione Lista dei Membri

Descrizione: Questo caso d'uso permette all'utente di visualizzare la lista dei membri del gruppo.

Flusso Principale:

1. L'utente seleziona l'area User per visualizzare la lista dei membri.
2. Il sistema mostra la lista dei membri del gruppo.

3.3.7 CU7-Visualizzazione Lista di Spese e Specifiche

Descrizione: Questo caso d'uso permette all'utente di visualizzare la lista di spese del gruppo e le relative specifiche.

Flusso Principale:

1. L'utente seleziona l'area Expense per visualizzare la lista di spese.
2. Il sistema mostra la lista di spese del gruppo ordinate dalla data meno recente alla più recente.
3. L'utente può selezionare una spesa per visualizzarne le specifiche.

3.3.8 CU8-Creazione Spesa

Descrizione: Questo caso d'uso permette all'utente di inserire una nuova spesa.

Flusso Principale:

1. L'utente accede all'area Expense.
2. L'utente seleziona l'opzione per aggiungere una nuova spesa.
3. Il sistema richiede i dettagli della nuova spesa.
4. L'utente inserisce i dettagli della spesa.
5. Il sistema verifica e aggiunge la nuova spesa.
6. Il sistema comunica la conferma dell'inserimento.
7. Il sistema aggiorna la lista delle spese del gruppo.

3.3.9 CU9- Aggiunta membro

Descrizione: Questo caso d'uso permette all'amministratore di aggiungere un utente al gruppo.

Flusso Principale:

1. L'amministratore seleziona l'area Users.
2. L'amministratore seleziona il bottone "New User".
3. Il sistema restituisce la lista di utenti non appartenenti al gruppo.
4. L'amministratore seleziona un utente.
5. Il sistema genera un alert.
6. L'amministratore conferma l'aggiunta dell'utente.
7. Il sistema aggiorna la lista di utenti non appartenenti al gruppo.

3.4 Architettura

L'architettura utilizzata per lo sviluppo dell'applicazione Piggy Bank in Flutter segue ugualmente il pattern MVVM (Model-View-ViewModel). Le classi legate al **Model** sviluppate per l'applicazione sono le seguenti.

La classe *GroupUser.dart* rappresenta un utente dell'applicazione e contiene i seguenti campi:

- *id*: identificatore univoco ;
- *name*: nome dell'utente;
- *email*: indirizzo email dell'utente;
- *password*: password dell'utente;
- *userExpense*: lista di id delle spese sostenute dall'utente;
- *userPayment*: lista di id dei pagamenti sostenuti dall'utente.

La classe *Group.dart* rappresenta un gruppo all'interno dell'applicazione ed è formata dai seguenti campi:

- *id*: identificatore univoco;
- *name*: nome del gruppo;
- *type*: tipologia del gruppo;
- *members*: lista degli utenti appartenenti al gruppo;
- *groupExpense*: lista degli id delle spese e dei pagamenti sostenuti dai membri del gruppo;

- *idAmministratore*: identificatore univoco dell'utente creatore del gruppo.

La classe *Expense.dart* rappresenta una spesa sostenuta dagli utenti e contiene i seguenti campi:

- *id*: identificatore univoco;
- *name*: nome della spesa;
- *amount*: importo della spesa;
- *payer*: id dell'utente che effettua il pagamento;
- *debtors*: lista di id degli utenti che condividono la spesa;
- *date*: data in cui si effettua la spesa.

Le classi legate al **ViewModel** sono due *Groupviewmodel.dart* e *Grouprepository.dart* (Figura 14a e 14b).

```

1 import 'package:flutter/material.dart';
2 import 'package:mobile_app/model/group.dart';
3 import 'package:mobile_app/model/expense.dart';
4
5 import 'model/group.dart';
6 import 'group_repository.dart';
7
8 class GroupViewModel with ChangeNotifier {
9   final GroupRepository groupRepository = GroupRepository();
10   late List<Group> _groups = [];
11   late List<GroupUser> _groupUsersInGroup = [];
12   late List<Expense> _expenses = [];
13   late List<GroupUser> _groupUsersInGroup = [];
14   bool _isCurrentUserAdmin = false;
15
16
17
18   List<Group> get groups => _groups;
19   List<GroupUser> get groupUsersInGroup => _groupUsersInGroup;
20   List<Expense> get expenses => _expenses;
21   bool get _isCurrentUserAdmin => _isCurrentUserAdmin;
22
23
24   // Metodo per caricare i dati dal database
25   Future<void> loadGroups() async {
26     try {
27       final groups = await groupRepository.getGroupsForCurrentUser();
28       _groups = groups;
29       notifyListeners();
30     } catch (e) {
31       print('Error in loadGroups: $e');
32     }
33   }
34 }

```

(a) GroupRepository.dart

```

1 import 'model/group.dart';
2
3 class GroupRepository {
4   final CollectionReference groupsCollection =
5     FirebaseFirestore.instance.collection('groups');
6   final FirebaseFirestore _auth = FirebaseFirestore.instance;
7   final CollectionReference usersCollection =
8     FirebaseFirestore.instance.collection('users');
9   final CollectionReference expensesCollection =
10     FirebaseFirestore.instance.collection('expenses');
11
12   Future<List<Group>> getGroupsForCurrentUser() async {
13     try {
14       final User? currentUser = _auth.currentUser;
15       final userId = currentUser?.uid;
16
17       final query =
18         groupsCollection.where('creatorId', arrayContains: userId).get();
19       final querySnapshot = await query;
20
21       return querySnapshot.docs
22         .map((document) => Group.fromDocument(document))
23         .toList();
24     } catch (e) {
25       print('Error in getGroupsForCurrentUser: $e');
26       return [];
27     }
28   }
29 }

```

(b) Esempio di funzione nel repository

Figura 16: Due istantanee della classe Grouprepository.dart.

Le classi legate alla componente **View** sono le seguenti:

- Addexpense.dart
- Adduser.dart
- Creategroup.dart
- Detailsgroup.dart
- Expensedetail.dart

- Listgroup.dart
- Loginfragment.dart
- Registerfragment.dart
- Userdetail.dart

3.5 Database

Lo sviluppo dell'applicazione in Flutter fa uso del servizio Firebase Firestore e Firestore Authentication. La struttura del database dell'applicazione è formata dalle seguenti raccolte :

- *expenses*;
- *user*;
- *groups*.

Le raccolte ed i loro campi coincidono con le classi di modello dell'applicazione (Figura 15).

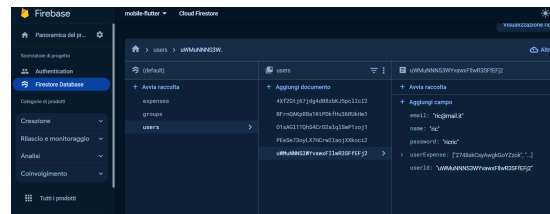


Figura 17: Firestore Database

3.6 UI

3.6.1 Mockup

3.6.2 Rappresentazione grafica dei casi d'uso

In questo paragrafo verranno esposti alcuni tra i casi d'uso più rilevanti dell'applicazione.

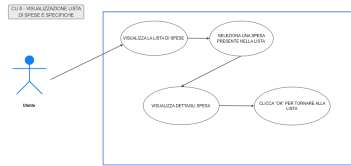


Figura 18: CU-8 Visualizzazione lista di spese e specifiche

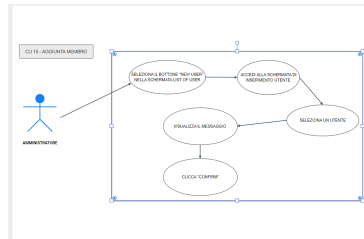


Figura 19: CU-15 Aggiunta membro

3.7 Sviluppo

Lo sviluppo dell'applicazione in Flutter ha previsto lo stesso approccio metodico utilizzato per quella in Android. Per quanto riguarda l'organizzazione (Figura 20), dato il numero considerevolmente inferiore di classi, è presente un solo package "model" che include le sole classi di modello.

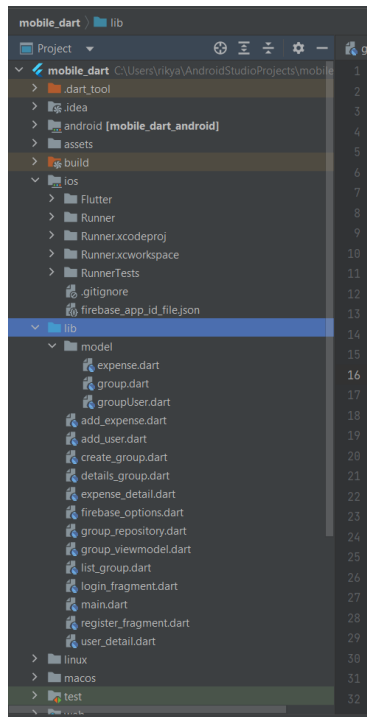


Figura 20: Organizzazione dell'applicazione flutter

4 Conclusioni

In conclusione, il progetto Piggy Bank è il frutto di un'elaborazione di idee mirate a realizzare un'applicazione in grado di semplificare la gestione delle spese condivise, mantenendo al contempo un'esperienza utente intuitiva e accessibile. Durante il percorso di sviluppo, la metodologia step-by-step adottata ha consentito l'implementazione graduale di varie funzionalità, garantendo, al contempo, una verifica accurata del corretto funzionamento delle varie sezioni dell'app. La rappresentazione grafica del mockup dell'interfaccia e dei casi d'uso ha contribuito a fornire una visione chiara delle interazioni previste. La realizzazione del progetto è stata resa possibile grazie all'insieme di conoscenze acquisite nel corso di Programmazione Mobile, integrate da un lavoro individuale di ricerca e approfondimento.